TaskTracker — Desafio Cumulativo (Fase 1)

Bootcamp II — Análise e Desenvolvimento de Sistemas

SEÇÃO 1 — IDENTIFICAÇÃO E ENTENDIMENTO DO PROBLEMA

1.1 Contextualização do Problema

No ritmo de vida atual, lidar com múltiplas responsabilidades ao mesmo tempo, trabalho, estudos, tarefas domésticas e compromissos pessoais, tornou-se a regra, não a exceção. Sem um método claro de organização, é comum que prazos sejam esquecidos, prioridades se confundam e a sensação de sobrecarga mental cresça, mesmo quando as tarefas em si não são complexas. Muitas pessoas ainda recorrem a anotações soltas em papel, aplicativos de notas genéricos ou simplesmente à memória, o que aumenta o risco de perder compromissos importantes e gera ansiedade desnecessária.

É nesse cenário que sistemas de gerenciamento de tarefas se tornam relevantes: eles não resolvem o problema de ter muita coisa para fazer, mas resolvem o problema de não saber o que fazer primeiro, quando e com que prioridade. A tecnologia, quando bem aplicada, pode transformar uma lista mental confusa em um processo estruturado, visível e controlável. O TaskTracker nasce dessa necessidade de oferecer uma ferramenta simples, acessível e objetiva para que qualquer pessoa , sem precisar de treinamento ou interfaces complicadas para que consiga organizar suas atividades diárias com clareza.

1.2 Definição do Sistema TaskTracker

O que é: o TaskTracker é uma aplicação de linha de comando (CLI), desenvolvida em Python, que permite ao usuário cadastrar, consultar, atualizar, concluir e remover tarefas diretamente pelo terminal.
Propósito principal: oferecer uma forma leve e direta de organizar tarefas do dia a dia, sem a necessidade de interfaces gráficas complexas ou dependência de internet.
Usuários-alvo: estudantes, profissionais autônomos e qualquer pessoa que já esteja familiarizada com o uso básico de terminal e queira uma ferramenta rápida de produtividade pessoal.

Funcionalidades essenciais (MVP):

1. Cadastrar nova tarefa
2. Listar todas as tarefas
3. Atualizar uma tarefa existente
4. Remover uma tarefa
5. Concluir uma tarefa
6. Filtrar tarefas por status

1.3 Diferencial do TaskTracker

O diferencial proposto para o TaskTracker é o foco em priorização visual e produtividade objetiva: em vez de apenas listar tarefas, o sistema destaca automaticamente tarefas vencidas e não concluídas, e permite filtrar rapidamente por status ou prioridade. Essa característica evita que o usuário precise garimpar manualmente entre dezenas de itens para saber o que é urgente. O próprio sistema evidencia isso na exibição da lista. É um diferencial simples, e que abre espaço, em fases futuras, para evoluções como notificações e estatísticas de produtividade.

SEÇÃO 2 — DECOMPOSIÇÃO DE DADOS E REGRAS

2.1 Estrutura de Dados da Tarefa

Campo	Tipo	Descrição	Exemplo	Obrigatório?
ID	Inteiro	Identificador único da tarefa	001	Sim (automático)
Título	String	Nome da tarefa	"Finalizar relatório"	Sim
Descrição	String	Detalhamento da tarefa	"Revisar os dados do setor"	Não
Datade Criação	Data/Hora	Momento do cadastro	08/09/2026 14:30	Sim (automático)
Datade Vencimento	Data	Prazo final	15/09/2026	Não
Prioridade	String	Nível de importância	Alta / Média / Baixa	Sim
Status	String	Situação atual	Pendente / Em andamento / Concluída	Sim
Categoria	String	Agrupamento temático	Trabalho / Estudos / Pessoal	Não
Data de Conclusão	Data/Hora	Momento em que a tarefa foi concluída	14/09/2026 18:10	Não (preenchido automaticamente ao concluir)

2.2 Entradas do Sistema

Entrada	Origem	Obrigatória?
Título da tarefa	Usuário	Sim
Descrição	Usuário	Não
Data de vencimento	Usuário	Não
Prioridade (Alta/Média/Baixa)	Usuário	Sim
Categoria	Usuário	Não
Opção escolhida no menu	Usuário	Sim
ID sequencial da tarefa	Sistema (automático)	—
Data/hora de criação	Sistema (automático)	—
Status inicial ("Pendente")	Sistema (automático)	—
Data/hora de conclusão	Sistema (automático, ao concluir)	—

2.3 Saídas do Sistema

Saída	Descrição
Listagem completa	Exibe todas as tarefas cadastradas, com ID, título, status, prioridade e data de vencimento
Filtros	Permite visualizar apenas tarefas Pendentes, Em andamento, Concluídas ou Todas
Destaques automáticos	Sinaliza visualmente tarefas vencidas (data de vencimento anterior à data atual e status diferente de "Concluída")
Resumos/estatísticas	Exibe a contagem de tarefas por status ao final da listagem geral (ex.: "3 pendentes, 1 em andamento, 5 concluídas")

2.4 Regras de Negócio

1. Toda tarefa deve ter, no mínimo, um título preenchido para ser cadastrada.
2. A data de vencimento, quando informada, não pode ser anterior à data atual.
3. A prioridade deve ser obrigatoriamente uma das três opções pré-definidas: Alta, Média ou Baixa.
4. O status inicial de toda tarefa recém-cadastrada é sempre "Pendente".
5. Ao concluir uma tarefa, o sistema deve registrar automaticamente a data e hora de conclusão.
6. Tarefas com data de vencimento anterior à data atual e status diferente de "Concluída" devem ser destacadas como "vencidas" nas listagens.

7. O ID da tarefa é gerado automaticamente pelo sistema, de forma sequencial e não editável pelo usuário.
8. Não é permitido remover ou atualizar uma tarefa cujo ID não exista na base de dados — o sistema deve validar essa condição antes de qualquer operação.

SEÇÃO 3 — RECEITA LÓGICA (ALGORITMO EM PORTUGUÊS ESTRUTURADO)

3.1 Estrutura Geral do Algoritmo

O TaskTracker funciona em um ciclo contínuo de menu-ação-retorno: ao ser iniciado, o programa exibe uma mensagem de boas-vindas e, em seguida, entra em um laço de repetição que se mantém ativo enquanto o usuário não escolher a opção de sair. A cada iteração, o menu principal é exibido, o usuário escolhe uma opção numérica, e o sistema executa a rotina correspondente àquela escolha.

Cada uma das operações (cadastrar, listar, atualizar, remover, concluir e filtrar) segue uma lógica própria, mas todas compartilham o mesmo padrão: coleta de dados de entrada (quando necessário), validação das informações e retorno de uma mensagem clara ao usuário, seja de sucesso ou de erro. Esse padrão de repetição do menu principal garante que o usuário consiga realizar quantas operações desejar em uma única execução do programa, sem precisar reiniciá-lo a cada ação.

O encerramento do sistema só ocorre quando o usuário escolhe explicitamente a opção "Sair", momento em que uma mensagem de despedida é exibida e o laço principal é interrompido.

3.2 Algoritmo Passo a Passo

```
Algoritmo TaskTracker

INÍCIO
  Exibir mensagem de boas-vindas "TaskTracker - Seu Gerenciador de Tarefas"
  ENQUANTO verdadeiro FAÇA
    Exibir menu principal com opções:
    1 - Cadastrar nova tarefa
    2 - Listar todas as tarefas
    3 - Atualizar uma tarefa existente
    4 - Remover uma tarefa
    5 - Concluir uma tarefa
    6 - Filtrar tarefas por status
    7 - Sair do sistema

Ler opção escolhida pelo usuário

ESCOLHA opção

      CASO 1:
        Solicitar título da tarefa
        Solicitar descrição (opcional)
```

```
    Solicitar data de vencimento (opcional)
    Solicitar prioridade (Alta/Média/Baixa)
    Solicitar categoria (opcional)
    Gerar ID automaticamente
    Definir status como "Pendente"
    Registrar data/hora de criação
    Adicionar tarefa à lista de tarefas
    Exibir mensagem de confirmação

CASO 2:
    PARA cada tarefa na lista FAÇA
        Exibir: ID, Título, Status, Prioridade, Data de Vencimento
    FIM PARA
    Exibir resumo com totais por status

CASO 3:
    Exibir lista de tarefas com IDs
    Solicitar ID da tarefa a ser atualizada
    SE tarefa existir ENTÃO
        Solicitar novo título (ou manter)
        Solicitar nova descrição (ou manter)
        Solicitar nova data de vencimento (ou manter)
        Solicitar nova prioridade (ou manter)
        Solicitar nova categoria (ou manter)
        Atualizar dados da tarefa
        Exibir mensagem de confirmação
    SENÃO
        Exibir mensagem de erro "Tarefa não encontrada"
    FIM SE

CASO 4:
    Exibir lista de tarefas com IDs
    Solicitar ID da tarefa a ser removida
    SE tarefa existir ENTÃO
        Confirmar remoção (Sim/Não)
        SE confirmado ENTÃO
            Remover tarefa da lista
            Exibir mensagem de confirmação
        SENÃO
            Exibir mensagem de operação cancelada
        FIM SE
    SENÃO
        Exibir mensagem de erro "Tarefa não encontrada"
    FIM SE

CASO 5:
    Exibir lista de tarefas pendentes com IDs
```

```
        Solicitar ID da tarefa a ser concluída
        SE tarefa existir ENTÃO
          SE status da tarefa for "Concluída" ENTÃO
            Exibir mensagem "Tarefa já está concluída"
          SENÃO
            Alterar status para "Concluída"
            Registrar data de conclusão
            Exibir mensagem de confirmação
          FIM SE
        SENÃO
          Exibir mensagem de erro "Tarefa não encontrada"
        FIM SE

      CASO 6:
        Exibir opções de filtro:
        1 - Pendentes
        2 - Em andamento
        3 - Concluídas
        4 - Todas
        Ler opção de filtro
        Exibir tarefas que correspondem ao filtro selecionado

      CASO 7:
        Exibir mensagem de encerramento "Obrigado por usar o TaskTracker!"
        SAIR do programa

      CASO CONTRÁRIO:
        Exibir mensagem de erro "Opção inválida! Tente novamente."
    FIM ESCOLHA

  FIM ENQUANTO
FIM
```

3.3 Explicação do Fluxo

O fluxo de dados do TaskTracker é centrado em uma estrutura de repetição (ENQUANTO) que mantém o programa ativo até uma decisão explícita de saída, e uma estrutura de seleção (ESCOLHA) que direciona a execução conforme a opção escolhida. As decisões condicionais (SE... ENTÃO... SENÃO) aparecem principalmente nas operações que dependem da existência prévia de uma tarefa — atualizar, remover e concluir —, garantindo que o sistema nunca tente manipular um dado inexistente. Esse encadeamento de validações é o que torna o algoritmo seguro e previsível, mesmo em um projeto simples: cada entrada do usuário é verificada antes de gerar uma saída ou alterar o estado interno da lista de tarefas.

SEÇÃO 4 — MAPEAMENTO ARQUITETURAL (GIT/GITHUB)

4.1 Estrutura de Pastas do Repositório

```
tasktracker/
│
├── src/
│   ├── __init__.py
│   ├── main.py
│   ├── task_manager.py
│   └── utils.py
│
├── docs/
│   ├── planejamento.md
│   └── manual_usuario.md
│
├── tests/
│   └── test_task_manager.py
│
├── .gitignore
├── README.md
└── requirements.txt
```

4.2 Descrição de Cada Pasta/Arquivo

src/: contém todo o código-fonte da aplicação.

- __init__.py: indica que a pasta src é um pacote Python, permitindo importações organizadas entre os módulos.
- main.py: ponto de entrada do programa; é o arquivo executado para iniciar o TaskTracker e exibir o menu principal.
- task_manager.py: concentra a lógica de negócio — funções de cadastrar, listar, atualizar, remover, concluir e filtrar tarefas.
- utils.py: reúne funções auxiliares e reutilizáveis, como validações de data e formatação de exibição no terminal.

docs/: documentação de apoio ao projeto.

- planejamento.md: registra as decisões de planejamento (a própria base deste documento evolui para cá).
- manual_usuario.md: explica, em linguagem simples, como usar o sistema.

tests/: contém os testes automatizados.

- test_task_manager.py: testes unitários das funções de task_manager.py, garantindo que as regras de negócio continuem funcionando corretamente conforme o código evolui.

.gitignore: lista arquivos e pastas que não devem ser versionados (ex.: ambientes virtuais, arquivos de cache).

README.md: apresenta o projeto, suas funcionalidades e instruções de instalação/uso — geralmente o primeiro arquivo lido por quem acessa o repositório.
requirements.txt: lista as dependências Python necessárias para rodar o projeto.

4.3 Estratégia de Versionamento com Git

Branches principais:

- main: contém sempre a versão estável e funcional do projeto.
- develop: branch de integração, onde novas funcionalidades são reunidas antes de seguir para a main.
- Branches auxiliares por funcionalidade (ex.: feature/cadastro-tarefa) podem ser criadas a partir de develop para isolar o desenvolvimento de cada recurso.

Commits semânticos: cada commit segue um padrão de prefixo que indica a natureza da alteração, por exemplo:

- feat: adicionar cadastro de tarefa
- fix: corrigir validação de data de vencimento
- docs: atualizar README com instruções de uso
- test: adicionar testes para conclusão de tarefa

Processo de push para o GitHub: o desenvolvimento ocorre localmente em uma branch de funcionalidade; ao concluir e testar a implementação, o código é enviado (push) para o repositório remoto e integrado à develop por meio de um Pull Request, que só é mesclado à main após revisão e testes passarem.

4.4 Justificativa Arquitetural

A separação entre src, docs e tests segue o princípio de responsabilidade única aplicado à organização de projetos: cada pasta tem uma finalidade clara e isolada, o que facilita tanto a manutenção quanto a escalabilidade do sistema. Ao separar a lógica de negócio (task_manager.py) das funções utilitárias (utils.py) e do ponto de entrada (main.py), o código fica mais fácil de testar, ler e evoluir, por exemplo, se no futuro o TaskTracker ganhar uma interface gráfica, apenas main.py precisaria ser adaptado, mantendo task_manager.py intacto. Essa estrutura também é compatível com o uso de containers (Docker), já previsto para a Fase 3, pois isola claramente o código-fonte das dependências e da documentação.

SEÇÃO 5 — APRESENTAÇÃO EM VÍDEO

LINK DO VÍDEO: https://youtu.be/fuqs5fU4puw